

ELEC 442

Lab 2

LEARNING GOALS

Inverse Kinematics (IK)

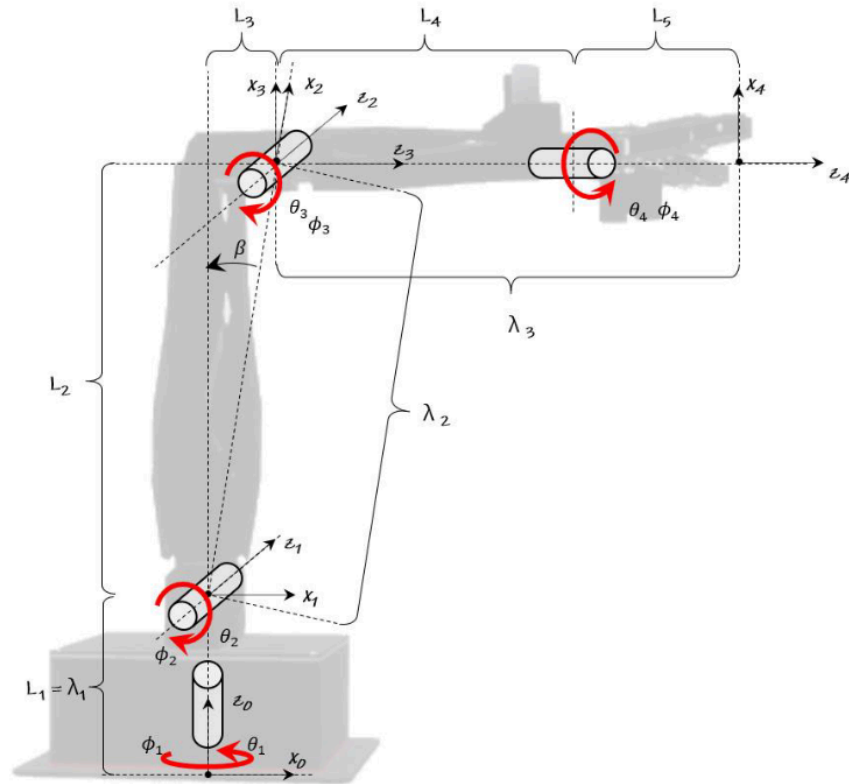
- **Handle Multi-Configuration Solutions:** Update an analytical IK solver to evaluate multiple valid joint configurations and programmatically select the optimal path based on tracking history.
- **Implement Numerical Solvers:** Build an iterative, Jacobian-based numerical root-finding solver (Newton-Raphson) for joint trajectory generation.
- **Analyze Convergence & Edge Cases:** Compare geometric vs. numerical methods regarding runtime performance, sensitivity to initial tracking states (ϕ_{prev}), and behavior near workspace boundaries.
- **Identify Kinematic Singularities:** Observe firsthand how approaching a singularity causes a standard numerical inverse Jacobian to fail or blow up.

Path Planning & Obstacle Avoidance

- **Build RRT Planners:** Implement a Rapidly-exploring Random Tree algorithm in joint space utilizing custom tree-management and collision-checking classes.
- **Tune Goal Biasing:** Configure and balance random sampling thresholds vs. goal-directed steps to optimize tree growth speed.
- **Implement Potential Fields:** Map 3D Cartesian forces acting on an end-effector into joint-space steering torques using the Jacobian Transpose matrix.
- **Troubleshoot Local Minima:** Identify structural failure modes where competing attractive and repulsive vectors trap the manipulator in dead zones, and propose algorithmic solutions to escape them.

1 Review

Welcome back! This is the second lab, focusing on inverse kinematics and continuing path planning. Before we get started let's do a quick review of the QArm.



Parameter	Value
L_1	0.14 m
L_2	0.35 m
L_3	0.05 m

Parameter	Value
L_4	0.25 m
L_5	0.15 m
β	8.13°

Figure 1: Quanser QArm Dimensions

Joint Specifications									
#	Item		Base	Shoulder	Elbow	Wrist	Units	Gripper	Units
1	Joint range	Min. Max	-170 +170	-85 +85	-95 +75	-160 +160	degrees	0 100	%
2	Max. speed		$\pm\pi/2$	$\pm\pi/2$	$\pm\pi/2$	$\pm\pi/2$	rad/s	± 100	%/s
3	Max. acceleration		$\pm\pi/3$	$\pm\pi/3$	$\pm\pi/3$	$\pm\pi/3$	rad/s/s	± 400	%/s/s
4	Current	Instantaneous Continuous	4.4 1.1	8.8 2.2	4.4 1.1	2.3 0.6	Amps	1.4 0.35	Amps
5	Torque	Instantaneous Continuous	10.6 2.65	21.2 5.3	10.6 2.65	4.1 1.0	Nm	1.9 0.5	Nm

Figure 2: Quanser QArm Joint limits

2 Inverse Kinematics

It should be clear why we are motivated to explore inverse kinematics (IK). When we have a target or destination, we are generally able to measure and define its coordinates within a fixed frame of reference. The challenge is commanding our manipulator, with its specific arrangement of joints and links, to reach that target. As the name implies, we are doing the exact opposite of forward kinematics: mapping a desired position from task space back to configuration space (joint values).

There are two primary ways to approach this problem:

- The Geometric Approach: Given the relatively straightforward geometry of our arm, an analytical or geometric approach is highly effective. This allows us to derive algebraic equations that yield a deterministic, closed-form solution.
- The Numerical Approach: Alternatively, we can use an iterative numerical approach. This method leverages root-finding techniques (like the Newton-Raphson method) to perform gradient-based steps using the Jacobian matrix you implemented in the previous lab. While the numerical approach can be less predictable and computationally heavier, it is far more generalizable and becomes essential when an analytical solution is difficult or impossible to derive.

2.1 Geometric Method

1. Setup and Intermediary Variables:

Given the end-effector position $p = (p_x, p_y, p_z)$ and link lengths $\lambda_1, \lambda_2, \lambda_3$:

$$A = \lambda_2$$

$$C = -\lambda_3$$

$$H = \lambda_1 - p_z$$

$$D = \pm \sqrt{p_x^2 + p_y^2}$$

2. Joint 3 (θ_3):

$$F = \frac{D^2 + H^2 - A^2 - C^2}{2A}$$

$$\theta_3 = 2 \arctan2(C \pm \sqrt{C^2 - F^2}, F)$$

3. Joint 2 (θ_2):

$$M = A + C \sin(\theta_3)$$

$$N = -C \cos(\theta_3)$$

$$\cos(\theta_2) = \frac{D \cdot M + H \cdot N}{M^2 + N^2}$$

$$\sin(\theta_2) = \frac{H - N \cos(\theta_2)}{M}$$

$$\theta_2 = \arctan2(\sin(\theta_2), \cos(\theta_2))$$

4. Joint 1 (θ_1):

Let the planar projection K be defined as:

$$K = \lambda_2 \cos(\theta_2) - \lambda_3 \sin(\theta_2 + \theta_3)$$

$$\theta_1 = \arctan2\left(\frac{p_y}{K}, \frac{p_x}{K}\right)$$

5. Final Mapping to QArm Joints (φ):

$$\varphi_1 = \theta_1$$

$$\varphi_2 = \theta_2 - \beta + \frac{\pi}{2}$$

$$\varphi_3 = \theta_3 + \beta$$

$$\varphi_4 = \gamma$$

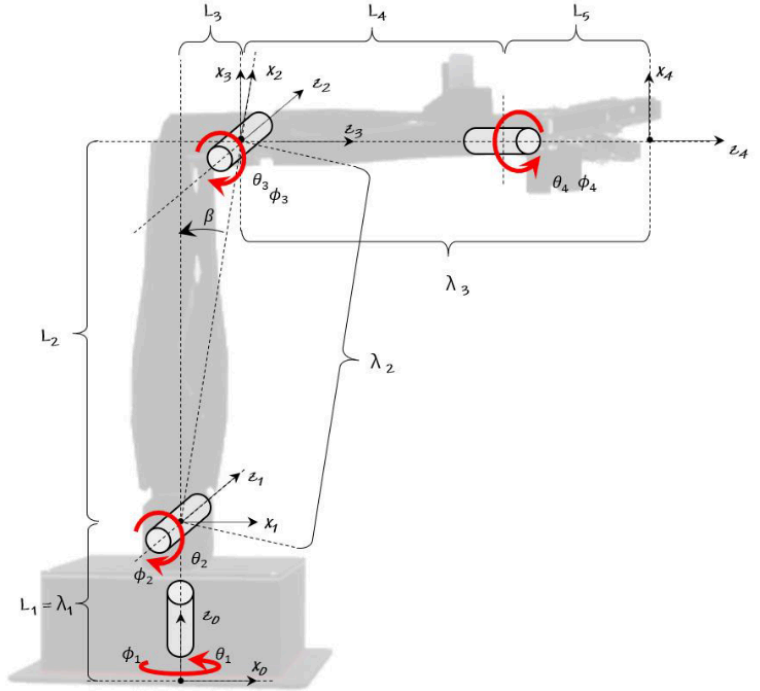


Figure 3: QArm Variables Reference

2.2 Implement Inverse Kinematics

2.2.1 Geometric Method (15 min)

While the geometric approach is conceptually straightforward, especially given the simple 3-DOF kinematic structure of the QArm, deriving the actual analytical expressions results in a fairly bulky set of equations. To save you some tedious algebra, these equations have already been implemented for you in the `geometric_inv_kin()` function below. Note that because the geometric approach involves solving trigonometric equations, both Joint 2 and Joint 3 yield two possible valid angles (e.g., elbow-up and elbow-down configurations). This results in a total of four distinct kinematic solutions for a given end-effector position.

```
allPhi, bestPhi = geometric_inv_kin(position, wrist, phi_prev)
```

Role	Name	Dimension	Description
input	phi_prev	4	Joint angles in order from base to wrist
	position	3	The desired position of the {4} origin in terms of the {0} coordinate system
	wrist_angle	1	the desired angle of the wrist joint in radians
output	allPhi	4×4	Matrix representing all 4 joint space solutions, each as a column
	bestPhi	4	The closest valid joint space solution given phi

Exercise 1:

At the moment, in `geometric_IK.py`, `geometric_inv_kin()` returns only `allPhi`. Edit `geometric_inv_kin()` so that we can pass in an array of joint positions as an argument (`phi_prev`), then the closest valid solution is outputted as `bestPhi`.

Description	Position (x, y, z)	wrist	phi_prev	allPhi	bestPhi
Normal	[0.3, 0.4, 0.35]	0.87	[0, 0, 0, 0]	$\begin{pmatrix} -2.21 & -2.21 & 0.93 & 0.93 \\ -2.14 & -0.49 & 0.2 & 1.86 \\ 0.11 & -2.97 & 0.11 & -2.97 \\ 0.87 & 0.87 & 0.87 & 0.87 \end{pmatrix}$	$\begin{pmatrix} 0.93 \\ 0.2 \\ 0.11 \\ 0.87 \end{pmatrix}$
Invalid wrist	[0.3, 0.4, 0.35]	3.49	[0, 0, 0, 0]	$\begin{pmatrix} -2.21 & -2.21 & 0.93 & 0.93 \\ -2.14 & -0.49 & 0.2 & 1.86 \\ 0.11 & -2.97 & 0.11 & -2.97 \\ -2.79 & -2.79 & -2.79 & -2.79 \end{pmatrix}$	Fail
Negative Position	[-0.2, -0.2, 0.6]	-2.62	[0, 0, 0, 0]	$\begin{pmatrix} 0.79 & 0.79 & -2.36 & -2.36 \\ -1.53 & 0.14 & -0.42 & 1.24 \\ 0.12 & -2.98 & 0.12 & -2.98 \\ -2.62 & -2.62 & -2.62 & -2.62 \end{pmatrix}$	$\begin{pmatrix} -2.36 \\ -0.42 \\ 0.12 \\ -2.62 \end{pmatrix}$
Out of workspace	[0.75, 0.3, 0.5]	0	[0, 0, 0, 0]	Fail	Fail
Singularity	[0, 0, 0.75]	0	[0, 0, 0, 0]	$\begin{pmatrix} -3.14 & 0. & -3.14 & 0. \\ -0.82 & 0.53 & -0.82 & 0.53 \\ -0.17 & -2.69 & -0.17 & -2.69 \\ 0. & 0. & 0. & 0. \end{pmatrix}$	Fail
Singularity in between	[0, -0.45, 0.5]	0	[0, 0, 0, 0]	$\begin{pmatrix} 1.57 & 1.57 & -1.57 & -1.57 \\ -1.79 & -0.28 & 0.00 & 1.51 \\ -0.03 & -2.83 & -0.03 & -2.83 \\ 0.00 & 0.00 & 0.00 & 0.00 \end{pmatrix}$	$\begin{pmatrix} -1.57 \\ 0 \\ -0.03 \\ 0 \end{pmatrix}$

Note: How you represent the fail condition is up to you, the recommendation is to return `phi_prev`

2.2.2 Numerical Method (15 min)

An alternative approach is to use an iterative numerical method. This is essentially a gradient-descent-style root-finding algorithm, where we leverage our ability to calculate the Jacobian to take successive steps from our current joint configuration toward the target position.

In its most general form, the update equation is expressed as:

$$\varphi^{i+1} = \varphi^i + \underline{J}^\dagger(\varphi^i) \begin{pmatrix} \varrho_d - \varrho_4(\varphi^i) \\ \theta(\varphi^i) \underline{s}(\varphi^i) \end{pmatrix}$$

For your implementation, you will be using the analytical Jacobian. Because the QArm's three positional degrees of freedom position the wrist, and the wrist itself only features a single rotational axis (wrist roll), the orientation tracking simplifies significantly. The full 6-DOF orientation error collapses into a single scalar value representing the wrist angle. Consequently, the analytical Jacobian simplifies to a 4×4 matrix (3 rows for x, y, z translation, and 1 row for the wrist rotation). Since this matrix is square, you can invert the Jacobian properly. We will also add a scalar step size scaling factor (α). The final result is:

$$\varphi^{i+1} = \varphi^i + \alpha \left[J^{-1}(\varphi^i) \begin{pmatrix} \mathcal{Q}_d - \mathcal{Q}_4(\varphi^i) \\ \theta(\varphi^i) \end{pmatrix} \right]$$

Where:

- φ is the (changing) vector of joint angles
- α is the step size scaling value
- J^{-1} is the inverse Jacobian
- $\begin{pmatrix} \mathcal{Q}_d - \mathcal{Q}_4(\varphi^i) \\ \theta(\varphi^i) \end{pmatrix}$ is the 4×1 error vector (3 rows for x, y, z position and the last row for wrist angle)

```
bestPhi = numeric_inv_kin(position, wrist, phi_prev)
```

Role	Name	Dimension	Description
input	phi_prev	4	Joint angles in order from base to wrist
	position	3	The desired position of the {4} origin in terms of the {0} coordinate system
	wrist_angle	1	the desired angle of the wrist joint in radians
output	bestPhi	4	The closest valid joint space solution given phi

Note: Be aware that if we approach a singularity, the inverse Jacobian will blow up

Exercise 2: In `numerical_IK.py`, Implement a numerical solver for the QArm using the provided `Inv_Jacobian()` function to get the inverse Jacobian. Several useful helper functions are listed in the python file as comments. Use the same test cases as before.

Description	Position (x, y, z)	wrist	phi_prev	bestPhi
Normal	[0.3, 0.4, 0.35]	0.87	[0, 0, 0, 0]	$\begin{pmatrix} 0.93 \\ 0.2 \\ 0.11 \\ 0.87 \end{pmatrix}$
Invalid wrist	[0.3, 0.4, 0.35]	3.49	[0, 0, 0, 0]	Fail
Negative Position	[-0.2, -0.2, 0.6]	-2.62	[0, 0, 0, 0]	$\begin{pmatrix} -2.36 \\ -0.42 \\ 0.12 \\ -2.62 \end{pmatrix}$
Out of workspace	[0.75, 0.3, 0.5]	0	[0, 0, 0, 0]	Fail
Singularity	[0, 0, 0.75]	0	[0, 0, 0, 0]	Fail
Singularity in between	[0, -0.45, 0.5]	0	[0, 0, 0, 0]	Fail

Note: since the numerical solver stops at a threshold, your values will be slightly different than the exact values provided by the geometric approach.

Note: How you represent the fail condition is up to you, the recommendation is to return `phi_prev`

Exercise 3: Compare the two solvers, what are their pros and cons?

A few key ideas to think about, but add your own ideas as well:

- Did one solver have trouble where the other did not?
- Can you think of reasons you would choose to use one solver or the other?
- How consistent are the solvers? (in terms of final position and time taken)

Exercise 4: Can you think of modifications that could improve each method?

LEARNING GOALS

Inverse Kinematics (IK)

- ✓ **Handle Multi-Configuration Solutions:** Update an analytical IK solver to evaluate multiple valid joint configurations and programmatically select the optimal path based on tracking history.
- ✓ **Implement Numerical Solvers:** Build an iterative, Jacobian-based numerical root-finding solver (Newton-Raphson) for joint trajectory generation.
- ✓ **Analyze Convergence & Edge Cases:** Compare geometric vs. numerical methods regarding runtime performance, sensitivity to initial tracking states (ϕ_{prev}), and behavior near workspace boundaries.
- ✓ **Identify Kinematic Singularities:** Observe firsthand how approaching a singularity causes a standard numerical inverse Jacobian to fail or blow up.

Path Planning & Obstacle Avoidance

- **Build RRT Planners:** Implement a Rapidly-exploring Random Tree algorithm in joint space utilizing custom tree-management and collision-checking classes.
- **Tune Goal Biasing:** Configure and balance random sampling thresholds vs. goal-directed steps to optimize tree growth speed.
- **Implement Potential Fields:** Map 3D Cartesian forces acting on an end-effector into joint-space steering torques using the Jacobian Transpose matrix.
- **Troubleshoot Local Minima:** Identify structural failure modes where competing attractive and repulsive vectors trap the manipulator in dead zones, and propose algorithmic solutions to escape them.

3 Path Planning - Obstacle Avoidance

In the previous lab, you implemented a cubic spline trajectory generator using differential kinematics. While this approach works perfectly in an unobstructed environment, real-world robotic workspaces are rarely completely empty. Furthermore, as you may have discovered in the extension exercises, moving blindly along a geometric trajectory can drive the manipulator into workspace boundaries or kinematic singularities.

To ensure safe and robust navigation, we must find a way to proactively avoid obstacles on the path to our destination. This lab introduces two fundamental path-planning paradigms: Artificial Potential Fields and Rapidly-exploring Random Trees (RRT).

The following sections will guide you through implementing each of these methods. To help you focus purely on the core algorithmic and conceptual design, several helper functions have been provided. A key component you will interact with is the `QArm_Lab_Interface` class, detailed below:

Method	Description	Inputs	Outputs
<code>write_to_arm(phi)</code>	Writes joint commands to the arm (in radians).	<code>phi</code>	-
<code>read_from_arm()</code>	Reads the current joint positions from the arm (in radians).	-	<code>phi</code>
<code>forward_kinematics(phi)</code>	Computes forward kinematics.	<code>phi</code>	<code>p4</code> (vector from base to EE in $\{0\}$ frame), <code>R4</code> (rotation matrix from $\{4\}$ to $\{0\}$ frame)
<code>Jacobian(phi)</code>	Computes the analytic 4×4 Jacobian.	<code>phi</code>	<code>J</code> (Jacobian matrix)
<code>Inv_Jacobian(phi)</code>	Returns the inverse of the analytic 4×4 Jacobian.	<code>phi</code>	<code>J_Inv</code> (Jacobian matrix inverse)

<code>inverse_kinematics(p, gamma, phi_prev)</code>	Computes inverse kinematics.	<code>p</code> (EE 3D coords), <code>gamma</code> (wrist angle), <code>phi_prev</code>	<code>phi</code> (all 4 possible solns.), <code>phiOptimal</code> (optimal soln.)
---	------------------------------	--	---

3.1 Obstacle Setup

Congrats on your good work on the “Risky Tube” exhibit! As the only members familiar with operating the QArm, you’ve now become too important at your job. Your next task is to calibrate the QArm inside the museum’s experimental metaphysics vault. The room contains massive, floor-to-ceiling cylindrical columns that look like cardboard, but they are actually the literal load-bearing pillars of reality.

If any part of the robotic arm touches these cylinders, all of reality will instantly collapse. Since you can’t just move in a straight line anymore, you’ll need to implement a way to navigate the workspace safely.

Don’t touch the pillars. That would be very bad.

Three obstacle setups will be used for both approaches. All startpoints and obstacles will be defined and labelled in the relevant python files (`obstacle_avoidance_potential.py` and `obstacle_avoidance_RRT.py`).

[Click here to see models in 3D](#)

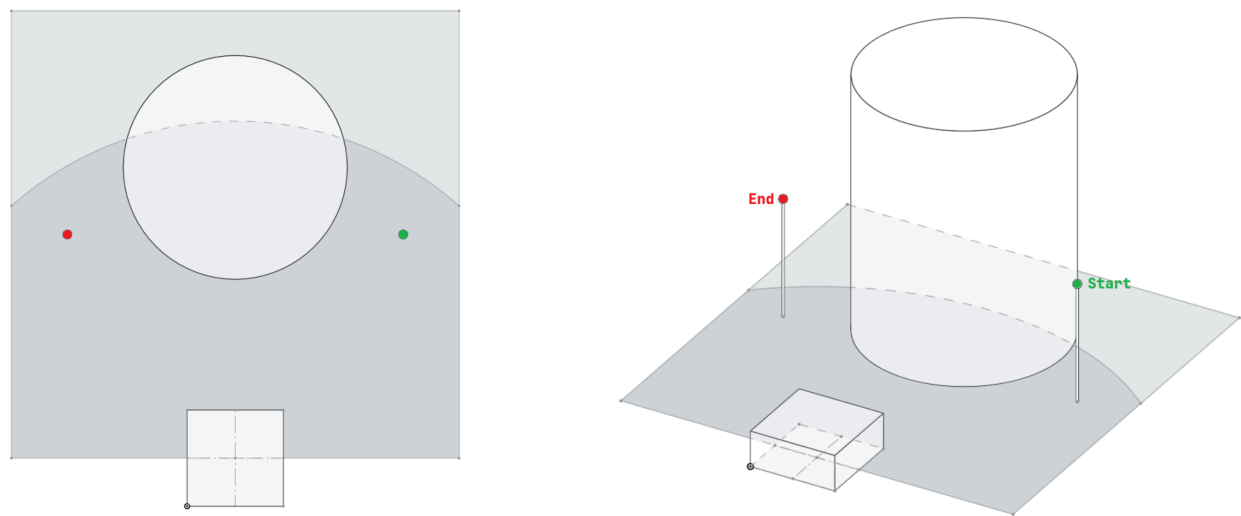


Figure 4: Obstacle Setup 1

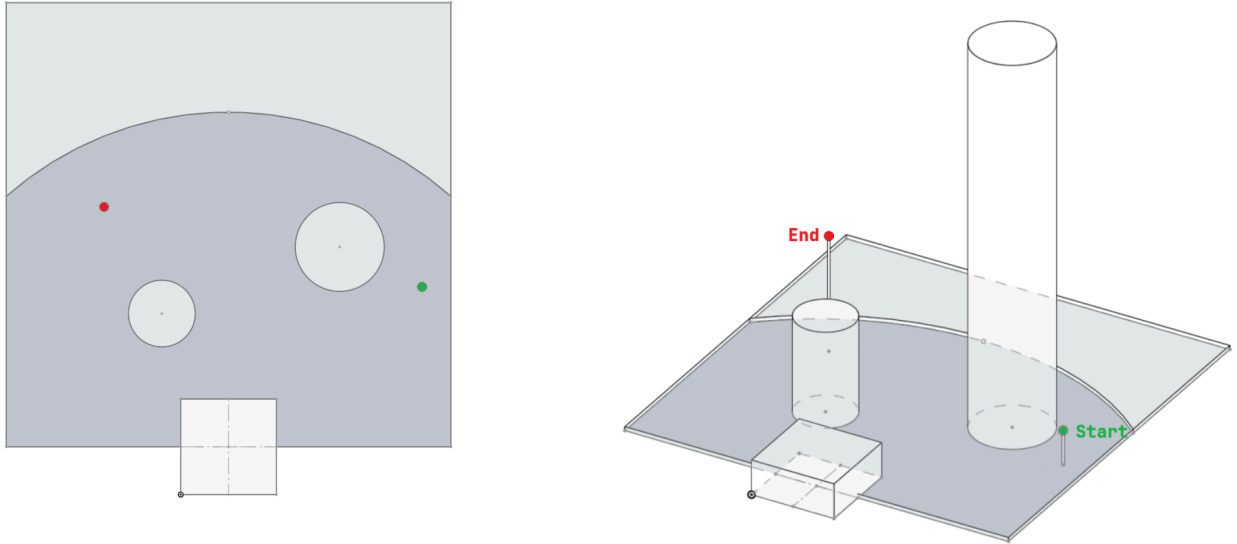


Figure 5: Obstacle Setup 2

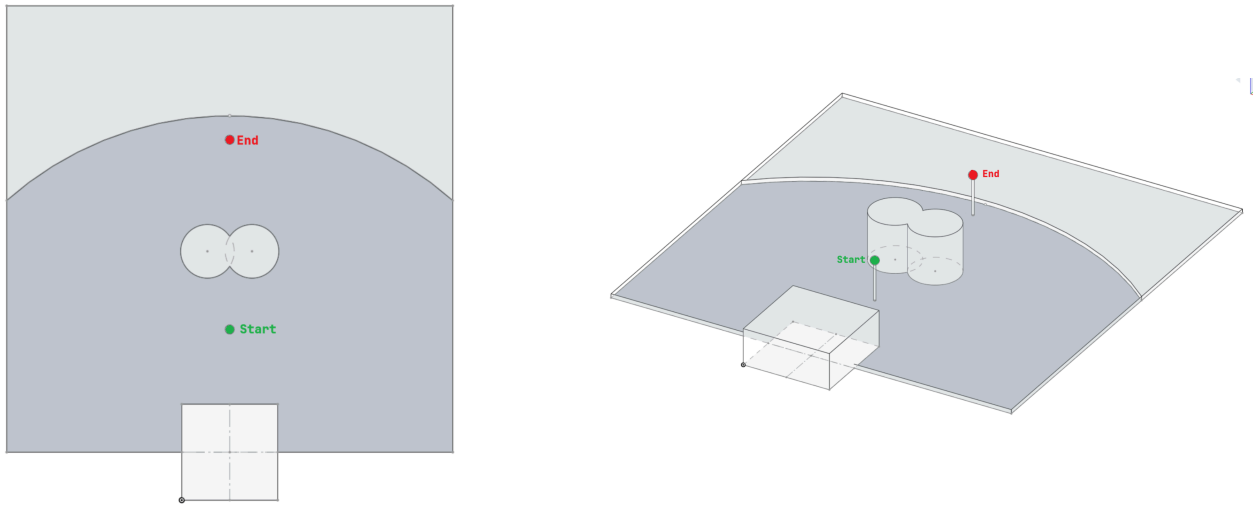


Figure 6: Obstacle Setup 3

3.2 Rapidly-exploring Random Tree (RRT) (45 min)

Rather than solving complex analytical path equations, RRT relies on an incremental sampling approach (LaValle, 1998). The algorithm builds a tree structure by taking small, discrete steps through the environment, verifying at each step that the configuration is collision-free before appending it to the tree. Because it randomly samples the space, RRT is probabilistically complete: as long as a valid path exists, the algorithm is guaranteed to find it eventually, though the resulting path is rarely the most optimal or efficient solution.

The planning process proceeds as follows:

1. Initialization: We begin with a known initial configuration and a target destination, both defined in joint space. The tree is initialized with the starting configuration as the root node.
2. Random Sampling: In each iteration, we choose a random coordinate within the allowable joint boundaries.
3. Goal Biasing: To prevent the tree from wandering aimlessly and to bias growth toward the goal, we introduce a goal-biasing probability. For instance, there is a 10% chance that we skip the random sample and choose the destination point directly as our target.
4. Extension: We locate the existing node in our tree that is closest to our sampled target point. We then take a small “step” of a fixed distance from that closest node toward the target.
5. Collision Checking: If the path segment spanning this step does not cause a collision, the new point is added to the tree as a child of the closest node.

After a sufficient number of iterations, a newly generated node will fall within an acceptable threshold distance of the destination. Once this target is reached, you can trace the path backward from this final node through its parent pointers all the way to the root. Reversing this sequence yields a continuous, collision-free path from start to finish.

Exercise 5: Create an RRT algorithm in the file `obstacle_avoidance_RRT.py`.

Note: be careful with step sizes, make it too large and you may “skip” through an obstacle

Test and demonstrate your program in all obstacle setups.

To help you manage a tree, the `RRT_Tree_Manager` has been implemented. It will track nodes and keep a parallel list to track parents. It also has the following functions:

Methods	Description	Inputs	Outputs
<code>add_node(q_new, parent_index)</code>	Adds a node to the list, and records its parent node	<code>q_new, parent_index</code>	-
<code>get_nearest_neighbor(q_rand)</code>	Finds the node closest to <code>q_rand</code>	<code>q_rand</code>	<code>node, node_idx</code>
<code>get_size()</code>	Returns the number of nodes in the tree	-	<code>size</code>

The `RRT_helpers` class is implemented with the following functions to help with implementation:

Methods	Description	Inputs	Outputs
<code>is_configuration_colliding(phi)</code>	checks if the <code>phi</code> is valid and avoids collision with obstacles	<code>joint_angles</code>	True if invalid position, False otherwise

3.3 Artificial Potential Fields (45 min)

The Artificial Potential Fields (APF) method treated in this lab operates on the virtual force framework (Khatib, 1986). In this technique, we treat the robot’s workspace as an energy landscape dictated by virtual potential fields. The target destination acts as a global minimum, generating an attractive potential that pulls the robot inward. Conversely, obstacles act as high-energy peaks, generating a repulsive potential that pushes the robot away.

To keep this exercise manageable, we have introduced a few simplifications:

- **End-Effector (EE) Control Only:** Your implementation will calculate the virtual forces acting strictly on the end-effector. In a production-grade implementation, potential fields are typically evaluated across intermediate joints and links to prevent the body of the arm from colliding with obstacles. Doing so, however, requires evaluating complex geometric hulls and computing multiple task-space Jacobians.
- **Disregarding Wrist Orientation:** Because the wrist joint only controls the orientation of the end-effector and does not alter its 3D Cartesian position, we will ignore its contribution to the positional planning.

While Khatib took a dynamics approach, using the transpose of the Jacobian and the principle of virtual work to relate fictitious forces ($\underline{F}$) to joint torques ($\underline{\tau}$): $\underline{\tau} = \mathbf{J}^T \underline{F}$ we will take a different route. Instead, we will rely on your differential kinematics knowledge and use the inverse Jacobian to compute joint velocities:

$$\dot{\underline{q}} = \mathbf{J}^{-1} \underline{F}$$

Here, we treat the fictitious force vector as a directional guide, since it directly represents the task space corrective movement we need to make. By carefully choosing our fictitious force formulae, this movement scales appropriately. However, a maximum value must be defined to prevent excessive, potentially dangerous joint velocities.

Once we have our joint space velocity ($\dot{\underline{q}}$), we adjust our trajectory just as we did before:

$$\varphi_{\text{new}} = \varphi_{\text{prev}} + \frac{\dot{\underline{q}}}{\|\dot{\underline{q}}\|}$$

Note: It is important to understand that φ_{new} and φ_{prev} represents a virtual or reference configuration. When you send this updated command to the hardware, the physical arm will naturally “lag” slightly behind this virtual target as the motors work to catch up. Because of this, you must include a brief sleep (delay) at the end of your program. This gives the physical arm the time it needs to settle into its final destination before the script terminates.

In order to convert distances to the obstacles (provided to you) and the destination point into fictitious forces, we use the following forumlae (adapted from Khatib, 1986):

$$\underline{F}_{\text{attractive}} = \begin{cases} -\zeta(\underline{p}_4 - \underline{x}_{\text{goal}}) \cdot \frac{\underline{p}_4 - \underline{x}_{\text{goal}}}{\|\underline{p}_4 - \underline{x}_{\text{goal}}\|} & \text{if } \|\underline{p}_4 - \underline{x}_{\text{goal}}\| \leq d_0 \\ -\zeta d_0 \cdot \frac{\underline{p}_4 - \underline{x}_{\text{goal}}}{\|\underline{p}_4 - \underline{x}_{\text{goal}}\|} & \text{if } \|\underline{p}_4 - \underline{x}_{\text{goal}}\| > d_0 \end{cases} \quad \text{where } \underline{p}_4 \text{ is the EE position in 3D}$$

$$\underline{F}_{\text{repulsive}} = \begin{cases} \eta \left(\frac{1}{\rho} - \frac{1}{\rho_0} \right) \frac{1}{\rho^2} \cdot \hat{u} & \text{if } \rho \leq \rho_0 \\ 0 & \text{if } \rho > \rho_0 \end{cases} \quad \text{where } \rho \text{ is distance from the obstacle to EE}$$

$$\underline{F}_{\text{total}} = \underline{F}_{\text{attractive}} + \underline{F}_{\text{repulsive}}$$

$$\underline{F} = \min(\|\underline{F}_{\text{total}}\|, \alpha) \hat{F}_{\text{total}}$$

where:

- ζ and η are the attractive and repulsive scaling factors, respectively.
- d_0 and ρ_0 are the threshold distances for clamping the forces.
- $\hat{u}$ is the unit vector pointing from obstacle to the end-effector
- α is the maximum allowable fictitious force magnitude (which dictates your maximum velocity and, consequently, your step size).

Exercise 6: In `obstacle_avoidance_potential.py` implement obstacle avoidance using potential fields. All variables have been assigned a starting value that will be roughly correct, once implemented, do some tuning to improve performance.

Test and demonstrate your program with all obstacle setups.

The `Potential_Fields_helper` class is implemented with the following functions to help with implementation:

Methods	Description	Inputs	Outputs
<code>get_all_obstacle_distances(self, p4)</code>	returns a list of dictionaries containing distances and directions (EE to obstacle)	<code>p4</code> (EE position in {0} system)	<code>obstacle_data</code>

Tips:

- use `J = QArm_Interface.Iv_Jacobian(phi)` to get the inverse of the 4×4 analytic Jacobian.
 - Don't forget to append a zero to the force vector before multiplying (for the wrist that we are ignoring)
- Use `QArm_Interface.write_to_arm(phi)` to move the arm to the joint positions specified by `phi`
- Make sure to add a slight delay at the end of your program to allow time for the arm to settle

Exercise 7: Now that you have experienced both methods, list some advantages and disadvantages each method has.

A few key ideas to think about, but list more if you think of anything:

- How “fast” are they? (Assuming similar step sizes, but including processing times)
- Can the manipulator get “stuck”? Why? (Assuming it is possible to reach the destination)
- How would they handle an environment with (predictably) moving obstacles?
- How would they scale to more complex obstacles and environments?
- How would they scale to more complex manipulators?

Exercise 8: What are some improvements you can think of to make to each method?

A few key ideas to think about, but don’t stop there:

- How could you adapt them to work in unpredictable environments? (Assuming you had some sort of live map of the environment)
- Could you improve the way each method moves (or is unable to move)?



PUSH CHANGES TO GITHUB

LEARNING GOALS

Inverse Kinematics (IK)

- ✓ **Handle Multi-Configuration Solutions:** Update an analytical IK solver to evaluate multiple valid joint configurations and programmatically select the optimal path based on tracking history.
- ✓ **Implement Numerical Solvers:** Build an iterative, Jacobian-based numerical root-finding solver (Newton-Raphson) for joint trajectory generation.
- ✓ **Analyze Convergence & Edge Cases:** Compare geometric vs. numerical methods regarding runtime performance, sensitivity to initial tracking states (ϕ_{prev}), and behavior near workspace boundaries.
- ✓ **Identify Kinematic Singularities:** Observe firsthand how approaching a singularity causes a standard numerical inverse Jacobian to fail or blow up.

Path Planning & Obstacle Avoidance

- ✓ **Build RRT Planners:** Implement a Rapidly-exploring Random Tree algorithm in joint space utilizing custom tree-management and collision-checking classes.
- ✓ **Tune Goal Biasing:** Configure and balance random sampling thresholds vs. goal-directed steps to optimize tree growth speed.
- ✓ **Implement Potential Fields:** Map 3D Cartesian forces acting on an end-effector into joint-space steering torques using the Jacobian Transpose matrix.
- ✓ **Troubleshoot Local Minima:** Identify structural failure modes where competing attractive and repulsive vectors trap the manipulator in dead zones, and propose algorithmic solutions to escape them.

4 Extension

Note: This section is not mandatory. Successful completion of the extension(s) will yield bonus marks.

4.1 Inverse Kinematics

4.1.1 Damped Inverse

One of the key challenges with numerical IK solvers is the risk of failure when operating close to a kinematic singularity. The issue is that the determinant of the Jacobian approaches zero, causing its inverse to spike toward infinity. One solution is to implement the Damped Inverse (Corke, 2023; section 8.3.3). This adds a damping factor to the Jacobian, preventing the determinant from reaching zero:

$$\dot{\underline{q}} = \left(\mathbf{J}(\underline{q}) + \lambda \mathbf{I} \right)^{-1} \underline{v}$$

Duplicate and modify your numerical IK solver to use the damped inverse. Test your implementation by including a singularity (e.g., placing the End Effector above the origin, $\{0\}$) in the path between `phi_prev` and the destination point (this is the case in the “singularity in between” test case).

4.2 Path Planning

4.2.1 RRT Path Smoothing

Implement a method to “smoothen” the trajectory generated by the RRT algorithm.

One common approach is to use a simple shortcutter. In this approach, we attempt to draw straight lines (in joint space) between two nodes. If no collisions are detected along that line, we can connect the two nodes directly and remove all intermediate nodes.

For a path with n nodes, where Node 1 is the starting position and Node n is the final position, the algorithm works as follows:

1. Attempt to draw a straight line from Node 1 to Node n .
2. Check multiple positions along this line using a predetermined step size.
3. If a collision is detected, stop and try again between Node 1 and Node $n - 1$.
4. Once a valid shortcut is found, the newly connected node becomes the new “Node 2,” and the process repeats using this node as the new starting point.

Ultimately, this should result in a much straighter path. Implement this behavior in a **copy** of your RRT Python file. Feel free to incorporate additional improvements (Another paper by LaValle (LaValle & Kuffner, 2001) covers several other ways to improve RRT).

4.2.2 Potential Fields Random Kick

As observed, the artificial potential fields method can cause the manipulator to get stuck in local minima (such as between two pillars). One simple solution is to monitor the velocity of the manipulator; if it remains low for a certain duration and the robot is not yet close to its target, apply a random “nudge” or kick.

Implement this logic in a **copy** of your artificial potential fields Python file. You are also welcome to experiment with a more sophisticated local-minima escape method.

5 Submission

Submit a single zip file containing all modified scripts:

- `geometric_IK.py`
- `numerical_IK.py`
- `obstacle_avoidance_RRT.py`
- `obstacle_avoidance_potential.py`
- Any extension files (if attempting bonus marks)
- Individual reflection (see below)

5.0.1 Individual Reflection

While the codebase is a collaborative group effort, **each student must submit a standalone, individual reflection document** (maximum 1 page).

1. Specify your group's workflow and the role each member played.
2. Detail at least one specific technical issue or unpredictable behavior your group encountered when running these algorithms. Explain how your group identified and resolved it.
3. Briefly describe everything covered in this lab and explain why the different approaches exist (what are they good/bad at) and some improvements that could be made.

Note: To prevent passive participation, TAs will cross-reference these notes against group submissions. Identical or closely paraphrased responses within a group will result in a grade of zero for the individual component.

6 References

- Corke, P. (2023). *Robotics, Vision and Control: Fundamental Algorithms in Python* (3rd ed.). Springer.
- Khatib, O. (1986). “Real-time obstacle avoidance for robot manipulators and mobile robots.” *The International Journal of Robotics Research*, 5(1), 90-98.
- LaValle, S. M. (1998). “Rapidly-exploring random trees: A new tool for path planning.” *Research Report 98-11*, Department of Computer Science, Iowa State University.
- LaValle, S. M., & Kuffner, J. J. (2001). “Rapidly-exploring random trees: Progress and prospects.” *Algorithmic and Computational Robotics: New Directions*, 293-308.
- Whitney, D. E. (1969). “Resolved Motion Rate Control of Manipulators and Human Prostheses.” *IEEE Transactions on Man-Machine Systems*, 10(2), 47-53.